THESIS PROGRESS DOCUMENT

PDF Workspace: State of Work Progress

A research-oriented PDF reading, indexing, review, and comparison system for turning document exploration into structured learning.

Project summary.

This repository implements an interactive research reading environment for PDFs. The system combines document viewing, question-answer chat, source-grounded indexing, spaced review, session-based research organization, and comparative analysis. The design goal is not only to help a reader consume papers, but to convert reading into durable, inspectable learning artifacts that can later support a knowledge graph.

Current thesis-facing contribution

The strongest current empirical contribution is the recall-first review pipeline: the reader answers from memory, Claude grades the response against source-grounded evidence, and FSRS scheduling is updated from the graded recall rather than self-rated confidence. This makes the review loop measurable as a learning intervention rather than only a note-taking feature.

Repository	`pdf-workspace`
Date	April 23, 2026
Primary stack	React, Vite, Zustand, FastAPI, SQLAlchemy, Postgres, ChromaDB, S3
LLM usage	Claude for chat, grading, ontology work, and comparative analysis
Research emphasis	Learning flow, retrieval practice, readable scientific interfaces, and research comparison
Current maturity	Working end-to-end system with active feature expansion and targeted UX repair

READER

INDEX

REVIEW

PDF viewer with TOC, source anchoring, selection actions, and HUD panels.

Saved highlights, synthesis, Q&A cards, concept tags, and review readiness.

Recall-first study session with Claude grading and FSRS scheduling.

SESSIONS

Research-session grouping layer above PDFs for organizational continuity.

COMPARE

Session-level comparative analysis driven primarily by AI reading of PDFs.

TESTING

Live Playwright checks focused on reading, indexing, review, and analysis flows.

1. RESEARCH PROBLEM AND RATIONALE

Why this system exists

The project addresses a common failure mode in academic reading tools: they are good at storage, search, and annotation, but weak at supporting comprehension, retrieval, and later comparison across papers. In practice, a reader can highlight extensively and still fail to retain what mattered, lose the context of a saved note, or struggle to compare one paper's claims against another.

This repository therefore treats reading as a staged learning pipeline rather than a passive viewing task. A document is uploaded, indexed, explored through chat and highlight actions, converted into study questions, reviewed under a recall-first protocol, and then situated inside a research session where multiple documents can be compared against a shared goal.

Core design rationale

- Reading support should reduce orientation cost without replacing the original paper.
- Saved questions must remain source-grounded so the user can audit what the system extracted.
- Review should measure recall quality, not only confidence or recognition.
- Comparative analysis should rely primarily on the source PDFs and only secondarily on reader-created material.
- Research organization should follow the user's inquiry structure, not only a flat PDF library.

Learning-science orientation

- **Retrieval practice:** the review loop asks for typed recall before revealing grading or source text.
- **Scaffolding:** chat, synthesis, term support, and session grouping reduce reader disorientation.
- **Progressive disclosure:** users can move from gist, to source, to deeper comparison.
- **Traceability:** study artifacts remain linked to passage, page, and paper context.
- **Comparative reasoning:** papers are eventually analyzed not as isolated texts but as competing or complementary interventions.

> **Practical thesis framing.** The app is not just a PDF reader with AI attached. Its thesis-relevant claim is that an interface can structure reading so that document interaction produces measurable learning signals, reusable study artifacts, and comparative research understanding.

Why the current work matters

The repository has already moved beyond a prototype that merely answers questions about a PDF. It now includes data models and workflows for storing knowledge artifacts, capturing review telemetry, organizing papers by research session, and generating session-level comparison outputs. This makes it suitable as a research platform rather than only a convenience layer over document chat.

2. END-TO-END LEARNING FLOW

How a reader moves through the system

The application is designed around a reader journey that begins with document ingestion and ends with study and comparison. This sequence is important because the project's architecture follows the learning flow closely.

Primary workflow

Upload PDF → read in viewer → select passage or ask chat question → log useful material to index → review saved Q&A cards → organize PDFs into research sessions → compare papers inside a session.

Reader mode

- PDF is displayed with page navigation and table-of-contents support.
- User can select text and invoke actions such as explain, simplify, key terms, summarise, and quiz.
- Selected passages are anchored to page and chunk metadata when possible.
- HUD tabs provide access to Chat, Index, and Compare while reading.

Chat and capture

- Chat answers are grounded in document excerpts when relevant.
- Short questions without selection can use a quick response mode.
- Useful responses can be logged to the index as study cards.
- Partial highlights are extended toward sentence boundaries to improve later readability.

Indexing and synthesis

- Saved entries store highlight text, section metadata, concepts, notes, synthesis, and nested Q&A cards.
- The index acts as a knowledge staging area between reading and review.
- Entries can be starred, flagged, anchored, reviewed, or merged.
- Deduplication protects the queue from near-duplicate study questions.

Review and retention

- Due cards are pulled from `qa_pairs` with FSRS state stored inline.
- Review begins with memory recall before grading feedback is shown.
- Claude grades recall on core claim, supporting detail, and faithfulness.
- Review logs preserve confidence, latency, reveal usage, grading output, and scheduler transitions.

Current learning-flow strength.

The system already supports a meaningful loop from reading to retrieval practice. The main active UX issue is contextual integrity: some saved questions become too detached from their source context in

the Index and Review flows. That is now an identified next-phase design target rather than an unknown problem.

3. CURRENT FEATURE SET

Implemented capabilities in the application

Feature area	Description
PDF ingestion	Upload PDFs, extract pages, chunk text, embed with MiniLM, store chunks in ChromaDB, and persist metadata in Postgres with S3-backed file storage.
Table of contents	Derive TOC from native outline when available, or fall back to font-size heuristics with debug support.
Selection actions	Explain, simplify, summarise, key terms, quiz, and chat-derived logging from passages or chat responses.
Chat assistant	Document-grounded assistant with layered response templates, quick mode for short questions, clickable page citations, and optional voice transcription.
Highlight index	Per-PDF index of saved passages, synthesis, notes, concepts, cluster tags, and study cards.
Deduplication	Near-duplicate study question detection with modal choices: open existing, merge answer, force save, or dismiss.
Review pipeline	Recall-first review overlay, confidence capture, source reveal telemetry, Claude grading, and FSRS scheduling.
Research sessions	Session-first left sidebar replacing a flat PDF library, with editable topic/context and manual PDF assignment.
Workspace tabs	Top-level page modes for Reader, Index, Compare, and Review, while keeping the HUD-style side panel in reader mode.
Comparative analysis	Session-level paper comparison driven mainly by AI analysis of PDF excerpts, with index/review material used as secondary anchors.
Testing	Playwright-based regression and exploratory scripts across upload, index, review, session organization, and compare flows.

Index-related capabilities

- Nested highlight entries with multiple Q&A pairs.
- On-demand synthesis and deeper synthesis storage fields.
- Concept extraction and topic tags.
- Star, flag, anchor, and review markers.

Session and compare capabilities

- Default sessions such as Learning Design Research and Medical Encyclopedia.
- Unsorted Research as an explicit inbox for new uploads.
- Suggestion flows for session naming and placement.
- AI literature-review matrix, custom axes, insight cards, gaps, and reader guidance.

4. COMPARATIVE ANALYSIS AND RESEARCH SESSION LAYER

How the app is evolving from single-paper reading to multi-paper research

A major recent architectural shift is the introduction of **research sessions** as a grouping layer above individual PDFs. This change matters because comparison, learning-graph construction, and session-level takeaways cannot be grounded in a flat library alone.

Research session model

- A `ResearchSession` stores a title, topic, context, ontology payload, and learning-takeaway payload.
- A many-to-many `ResearchSessionPDF` membership table supports future multi-session membership for one PDF.
- The legacy `pdf_documents.research_session_id` remains for compatibility and current primary-session behavior.

Why sessions matter for learning

- They frame reading around a research goal rather than a file list.
- They support later comparative tables and gap analysis.
- They separate unrelated corpora, such as learning-interface papers and medical reference texts.
- They create a future bridge toward session-level knowledge graphs.

Comparative analysis pipeline

The compare layer is implemented as a session-level service that samples PDF chunks directly from ChromaDB, generates per-paper analyses with Claude, synthesizes a comparison matrix, and caches the result on the research session. Reader-created index entries and Q&A cards are used as secondary anchors to show what the reader has already noticed, but the source PDFs remain primary evidence.

Fixed scholarly dimensions | Session-specific axes | Cross-paper insight cards | Research gaps | Reader decision guide

Current status.

The compare workspace now produces stronger custom axes and paper-to-paper contrasts than its first pass. It also includes reader-guidance cards that explain what each paper is best for, how it differs, and what to pair it with.

Current limitation.

The full value of comparison still depends on stronger organization in the Index and Review layers. If saved questions lose context, downstream comparison and future knowledge-graph work become noisier.

This session-first direction is important for the thesis because it repositions the system from “AI reader for one document” toward “research environment for accumulating, reviewing, and contrasting structured understanding across documents.”

5. ARCHITECTURE AND DATA MODEL

How the system is built

Layer	Current implementation
Frontend	React + Vite application with Zustand state store and dedicated workspace components for Reader, Index, Compare, and Review.
Backend API	FastAPI application with routers for PDFs, chat, highlights, review, research export, research sessions, and voice transcription.
Database	Postgres via SQLAlchemy ORM. Key entities include `pdf_documents`, `highlight_entries`, `qa_pairs`, `review_log`, `research_sessions`, and `research_session_pdfs`.
Vector retrieval	ChromaDB stores chunk embeddings and supports passage retrieval, chunk resolution, and PDF-first comparative analysis.
File storage	S3 stores uploaded PDFs, while the backend streams files and presigned URLs to the frontend.
Embeddings	SentenceTransformers `all-MiniLM-L6-v2` for chunk embeddings and deduplication similarity.
LLM functions	Claude models power chat generation, grading, ontology/facet work, study question preparation, and comparative analysis.
Testing	Playwright-based live and regression scripts plus migration/backfill verification utilities.

Key backend entities

- **PDFDocument:** title, S3 key, page count, chunk count, ontology, session pointer.
- **HighlightEntry:** passage, section data, concepts, synthesis, cluster tag, status flags.
- **QAPair:** question, study question, answer, source chunks, card type, FSRS fields, tags, provenance.
- **ReviewLog:** confidence, recall text, reveal telemetry, Claude grades, FSRS transition, versioning.
- **ResearchSession:** session topic, context, ontology, takeaways, and PDF memberships.

Architectural pattern

- The frontend store hydrates highlights and chat per selected PDF.
- The backend owns canonical study-question generation and dedup decisions.
- Review state lives inline on `qa_pairs` rather than in a separate memory-item table.
- Comparative analysis is cached on sessions to avoid recomputing every view.
- Schema evolution uses both Alembic and defensive boot-time `ALTER TABLE IF NOT EXISTS` statements.

> **Implementation philosophy.** The current codebase consistently favors traceability and compatibility: preserve stored artifacts, add features incrementally, and keep routes and models stable enough that new

> organizational layers do not erase prior work.

6. REPOSITORY STRUCTURE

Repo map and implementation organization

```
pdf-workspace/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── models/
│   │   │   ├── pdf.py
│   │   │   ├── highlight.py
│   │   │   ├── review.py
│   │   │   └── research_session.py
│   │   ├── routers/
│   │   │   ├── pdfs.py
│   │   │   ├── chat.py
│   │   │   ├── highlights.py
│   │   │   ├── review.py
│   │   │   ├── research.py
│   │   │   ├── research_sessions.py
│   │   │   └── voice.py
│   │   └── services/
│   │       ├── pdf_service.py
│   │       ├── toc_service.py
│   │       ├── s3_service.py
│   │       ├── chroma_service.py
│   │       ├── embedding_service.py
│   │       ├── chat_service.py
│   │       ├── card_service.py
│   │       ├── grading_service.py
│   │       ├── ontology_service.py
│   │       ├── research_session_service.py
│   │       └── comparative_analysis_service.py
│   ├── migrations/
│   ├── scripts/
│   └── tests/
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── store/index.js
│   │   ├── api/
│   │   └── components/
│   │       ├── PDFSidebar.jsx
│   │       ├── PDFViewer.jsx
│   │       ├── ChatPanel.jsx
│   │       ├── HighlightIndex.jsx
│   │       ├── ReviewSession.jsx
│   │       ├── WorkspaceIndex.jsx
│   │       ├── WorkspaceReview.jsx
│   │       └── WorkspaceCompare.jsx
├── playwright-tests/
├── docs/
├── chroma_data/
├── logs/
└── research/
```

What lives where

- **`backend/app/routers`** defines the external API surface.
- **`backend/app/services`** contains domain logic: ingestion, chat, grading, ontology, sessions, comparison.
- **`frontend/src/components`** implements the reader HUD and full-page workspaces.
- **`frontend/src/store`** coordinates selected PDF, sessions, chat history, highlight index, and review mode.
- **`playwright-tests`** captures live behavior and regression-sensitive learning flows.

Why this structure is useful for a thesis project

- Research logic is visible in service boundaries instead of being scattered through the UI.
- Testing artifacts are retained in the repository as methodological trace, not only CI scaffolding.
- The repo cleanly separates data model, interface shell, and AI-assisted analysis layers.
- The documentable units of work map well onto thesis chapters: interface, storage, retrieval, review, comparison.

State of work progress

Key shipped updates

- **Step 1:** canonical server-owned study-question schema and card-type contract.
- **Step 2:** dedup gate, soft deletion, merge flow, and write-time quality floor for study cards.
- **Step 3:** recall-first review UI with reveal telemetry and thesis-grade review logging.
- **Session-first sidebar:** transition from flat PDF library to research sessions.
- **Workspace tabs:** Reader, Index, Compare, and Review now have full-page modes.
- **Comparative analysis:** AI comparison cache now includes stronger custom axes, cross-paper insights, research gaps, and reader guidance.

Most important current limitation

The app now produces many useful study artifacts, but some saved questions still lose too much context once they leave the original chat or highlight moment. This creates friction in the Index and Review flows and weakens later organization for comparison and knowledge-graph work.

In short: the system is strong at producing learning material, but still needs better context preservation and organization for that material.

Current UX repair plan

- Normalize question context so every saved question carries provenance, scope, and a usable summary.
- Repair the right-side Index and Review flows so ambiguous questions remain answerable and traceable.
- Turn full-page Index and Review tabs into real deep-dive workspaces instead of enlarged side panels.
- Backfill weak legacy questions rather than silently deleting or rewriting them.

Near-term research value.

Even in its current state, the system already demonstrates a coherent pipeline from document reading to measurable review and session-level comparison. This is a credible state-of-work progress point for a thesis

Next phase.

The next phase is not adding another isolated feature. It is improving contextual integrity across Index, Review, and Session analysis so that all later work, especially comparative synthesis and eventual knowledge graphs, sits on better-organized learning artifacts.

because the architecture now supports both user-facing interaction and research logging.

Conclusion

The repository has evolved from an AI-assisted PDF viewer into a structured research reading environment. Its strongest present contribution is the integration of source-grounded reading support, recall-first review, and session-level comparison inside one coherent workspace. The system is operational, testable, and already suitable for methodological discussion as a thesis progress artifact, while clearly exposing the next design problem: preserving context and improving the organization of learned material.